Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 02 Luglio 2009

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ☐, Ghezzi ☐, San Pietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 8)

Si consideri il metodo statico **controllaMatrice**. Questo ha due parametri m e N, con m, una matrice quadrata di interi NxN (con N>2). Il metodo restituisce true nei casi seguenti:

-
- Ogni casella m[i][j] sopra la diagonale principale, e con indice j maggiore di 2, contiene la somma delle due celle precedenti della stessa riga. Ad esempio, m[3][4] = m[3][2] + m[3][3]
- Ogni casella m[i][j] sotto la diagonale principale, e con i maggiore di 2, contiene la somma delle due celle precedenti della stessa colonna. Ad esempio, m[3][1] = m[1][1] + m[2][1]
- I numeri sulla diagonale principale, a partire dal terzo, rispettano una regola analoga: ogni elemento è la somma dei due precedenti (sulla diagonale). Ad esempio m[4][4] = m[2][2] + m[3][3]

Si noti che i primi due elementi delle prime due righe sono valori arbitrari.

a- Si scrivano la pre- e post-condizione del metodo evidenziandone tutti gli elementi significativi

```
//@requires N > 2 &&
                m != null && m.length == N &&
                (\forall int i; 0 <= i && i < N; m[i] != null && m[i].length == N );
//@ensures (\result == true) <==>
                (\forall int i; 2 <= i && i < N; (\forall int j; 2 <= j && j < N;
                        ( (i == j) => (m[i][j] == m[i-1][j-1] + m[i-2][j-2]) ) &&
                        ( (i > j)  => (m[i][j] == m[i-1][j] + m[i-2][j]) ) &&
                        ( (i < j)  => (m[i][j] == m[i][j-1] + m[i][j-2]) )
                ));
```

public static boolean controllaMatrice(int[][] m, int N)

b- Si identifichino le eccezioni necessarie per rendere totale il metodo**, mostrando come cambia la specifica del metodo**.

```
//@requires true;
//@ensures   N > 2 &&
                 m != null && m.length == N &&
                 (\forall int i; 0 <= i && i < N; m[i] != null && m[i].length == N ) &&
               (\result == true) <==>
                 (\forall int i; 2 <= i && i < N; (\forall int j; 2 <= j && j < N;
                        ( (i == j) => (m[i][j] == m[i-1][j-1] + m[i-2][j-2]) ) &&
                        ( (i > j)  => (m[i][j] == m[i-1][j] + m[i-2][j]) ) &&
                        ( (i < j)  => (m[i][j] == m[i][j-1] + m[i][j-2]) )
               ));
//@signals (NullPointerException e) m == null ||
                                        (\exists int i; 0 <= i && i < N; m[i] == null);
//@signals (MatrixTooSmallException e) N <= 2;
//@signals (WrongMatrixException e) m != null &&
                                        ( m.length != N || (\exists int i; 0 <= i && i < N;
                                                m[i] != null && m[i].length != N) )
```

Esercizio 2 (punti 12)

L'azienda *happyCompany* deve preparare dei rendiconti per i propri dipendenti a progetto. Questo significa che ogni dipendente deve tenere traccia delle ore dedicate ogni giorno a ciascun progetto. Dopo vari tentativi rudimentali, *happyCompany* ha deciso di dotarsi di un'applicazione specifica.
Ogni **Dipendente** (si supponga che esista una classe con questo nome) è associato a un insieme di oggetti **RapportoGiornaliero** e questi vengono opportunamente aggiornati ogni mese. Ogni progetto corrisponde a un codice, che è un intero positivo, definito univocamente per quel progetto.

a- La classe **RapportoGiornaliero** ha una data, osservabile con un metodo **public Data data()**, e un metodo **public void aggiungiOra(int prg, int oi)** per segnare che il dipendente relativo a quel rapporto ha lavorato al progetto di codice **prg** nell'ora che inizia da **oi**. Ad esempio, **aggiungiOra(345, 9)** serve per dire che dalle 9 alle 9.59 la persona ha lavorato al progetto 345. Una persona può lavorare nella stessa ora ad un solo progetto per volta e non può lavorare per più di 8 ore al giorno. Per ragioni di flessibilità, ogni dipendente può spalmare il proprio lavoro come crede sulle 24 ore, ma senza mai eccedere il limite suddetto. *Si definiscano in JML la pre- e post-condizione* del metodo **aggiungiOra** sapendo che la classe **RapportoGiornaliero** offre anche un metodo **public int[] rendiconto**() che restituisce un array di 24 interi (ogni casella, una per ogni ora della giornata, contiene il codice del progetto su cui si e' lavorato, oppure 0 se l'ora in questione non è stata lavorata).
Si supponga che la classe Data abbia metodi giorno(), mese(), anno() con significato ovvio, che restituiscono ciascuno un int.

```
//@ requires prg != 0 && 0 <= oi && oi <= 23 &&
                        rendiconto()[oi] == 0 &&
                        (\numof int i; 0 <= i && i <= 23; rendiconto()[i] != 0) <= 7;
//@ensures rendiconto()[oi] == prg &&
                (\forall int i; 0 <= i && i <= 23;
                        (oi != i) => (rendiconto()[i] == \old(rendoconto()[i]) );
public void aggiungiOra(int prg, int oi)
```

b- La classe Dipendente offre i metodi:
public ArrayList<Integer> progetti(int mese, int anno) e
public ArrayList<RapportoGiornanaliero> rapporti(int mese, int anno)
che restituiscono rispettivamente l'insieme dei progetti su cui il dipendente ha lavorato nel mese **mese**, e l'insieme di tutti i rapporti giornalieri del mese per il dipendente, come ArrayList di **RapportoGiornaliero**,. Questi metodi servono per il consolidamento dei rapporti mensili, attraverso un ulteriore metodo della classe:
public void consolida(int mese, int anno)
che viene chiamato alla fine del **mese** per verificare che il dipendente non abbia lavorato più di 160 ore nel mese e abbia lavorato almeno 4 ore a ogni progetto dichiarato. Nel caso le due regole fossero violate, il metodo solleva le eccezioni **TroppoLavoroException** e **PocheOreException**, rispettivamente.

b1-*Si definiscano in JML la pre- e post-condizione, comprensive della gestione delle eccezioni,* per il metodo **consolida()**.

```
//@requires 0 <= mese && mese < 12;
//@ensures   (* il dipendente non lavora per più di 160 ore / mese *) &&
                \sum(int i; 0 <= i && i < rapporti(mese,anno).size();
                  \numof(int j; 0 <= j && j < 23;
                    rapporti(mese,anno).get(i).rendiconto()[j] != 0
                ) ) <= 160 &&
                (* il dipendente lavora almeno 4 ore ad ogni progetto *) &&
                \forall(int p; 0 <= p && p < progetti(mese,anno).size();
                  \sum(int i; 0 <= i && i < rapporti(mese,anno).size();
                    \numof(int j; 0 <= j && j < 23;
                      rapporti(mese,anno).get(i) .rendiconto()[j] ==
                      progetti(mese,anno).get(p)
                  )  ) >= 4
                );
//@signals (TroppoLavoroException e)
                \sum(int i; 0 <= i && i < rapporti(mese,anno).size();
                  \numof(int j; 0 <= j && j < 23;
                    rapporti(mese,anno).get(i) .rendiconto()[j] != 0
                ) ) > 160;
//@signals (PocheOreException e)
                \exists(int p; 0 <= p && p < progetti(mese,anno).size();
                  \sum(int i; 0 <= i && i < rapporti(mese,anno).size();
                    \numof(int j; 0 <= j && j < 23;
                      rapporti(mese,anno).get(i) .rendiconto()[j] ==
                      progetti(mese,anno).get(p)
                  )  ) < 4
                );
public void consolida(int mese, int anno)
        throws TroppoLavoroException, PocheOreException;
```

b2- *Si definiscano in JML la pre- e post-condizione* per il metodo **rapporti (mese, anno)**, assicurando la consistenza dei dati, e cioè che vengano restituiti solo i rapporti giornalieri del mese e dell'anno specificati, senza giorni duplicati.

Si ipotizza che chiedendo i rapporti per un giorno non lavorativo, venga restituita una lista vuota (non nulla)

```
//@ requires 0 <= mese && mese < 12;
//@ ensures  \result != null &&
                \forall(int g; 0 <= g && g < \result.size();
                   \result.get(g) != null &&
                   \result.get(g).data().anno() == anno &&
                   \result.get(g).data().mese() == mese &&
                   \forall(int z; g < z && z < \result.size();
                      \result.get(g).data().giorno() !=
                      \result.get(z).data().giorno()
                ) );
public /@ pure @/ ArrayList<RapportoGiornaliero> rapporti(int mese, int
anno);
```

b3- Invece di specificare la proprietà di cui al punto b2 mediante pre e post-condizione, si sarebbe potuto esprimere la stessa mediante un invariante pubblico? Si o no? Perchè?

Sintatticamente è possibile definire la postcondizione in un invariante, in quanto il metodo rapporti è un metodo puro, e le proprietà che vengono espresse validano la consistenza dell'intero oggetto. L'invariante pubblico esprimerebbe le stesse proprietà, definendo che, per ogni mese e per ogni anno, valgono le post-condizioni specificate.
Tuttavia, in JML occorre comunque che sia l'invariante ad essere garantito in base alla specifica dei metodi pubblici, e non viceversa. Quindi l'invariante non può semplicemente rimpiazzare la postcondizione.

c- *Happy Company* ha molto lavoro e deve ricorrere a consulenti esterni. Per questo, la classe **Consulente** viene aggiunta al sistema come sottoclasse di **Dipendente.** Il consulente però non ha limiti di orario: può lavorare quante ore vuole su qualunque progetto, senza ovviamente superare le 24 ore al giorno. Considerando l'ipotesi precedente, il metodo **consolida()**, ridefinito per la sottoclasse, rispetta il principio di sostituzione? Motivare brevemente la risposta.

NO

Il metodo **consolida()**, pur rispettando la regola delle signature, violerebbe la regola dei metodi. La specifica del metodo in *Dipendente* assicura che se questo termina correttamente, allora il dipendente ha lavorato meno di 160 ore al mese; in caso contrario viene sollevata l'eccezione *TroppoLavoroException*.
Il testo richiede di definire un *Consulente*, in cui vengono rimossi i vincoli di orario. Tuttavia, viene richiesto che *Consulente* sia sottoclasse di *Dipendente*. Questo comporta che, nel ridefinire il metodo *consolida*(), la classe *Consulente* deve mantenere le promesse fatte nel medesimo metodo dalla classe *Dipendente*.
Riassumendo, il metodo deve sia rimuovere i vincoli di orario (le nuove richieste) sia sollevare un'eccezione nel caso in cui il dipendente lavori più di 160 ore al mese. Si può notare che le due cose sono in palese contrasto, che è causato dalla violazione della regola dei metodi.

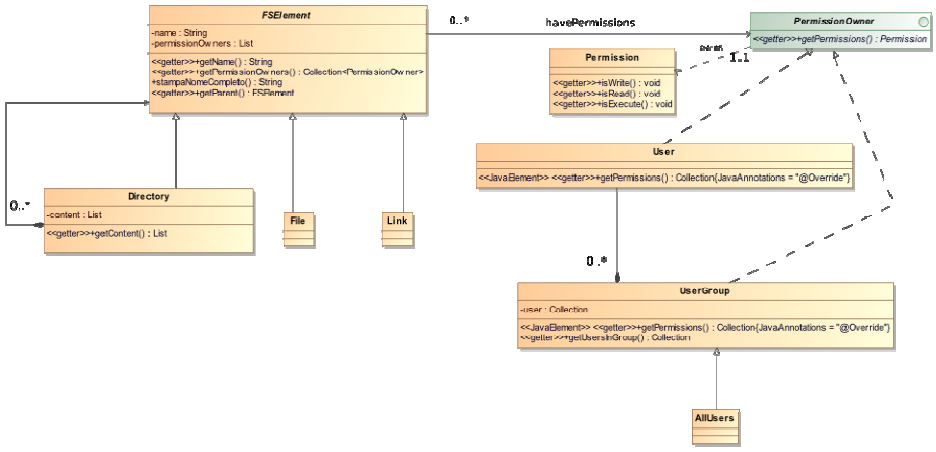
Esercizio 3 (punti 5)

Si consideri la struttura tipica di un file system. Le directory sono organizzate gerarchicamente: ogni directory può contenere altre directory, file, oppure link. Un link è un riferimento a un file fisicamente memorizzato in un'altra directory; in questo modo il file riferito diventa virtualmente parte anche della directory che contiene il link. Una directory ha un nome. Ogni file è caratterizzato da un nome, una dimensione e un tipo. Un link ha un nome e un tipo.
Ogni elemento (directory, file o link) è associato con un insieme di diritti d'accesso: lettura, scrittura e esecuzione. Questi sono concessi al proprietario di una risorsa (un singolo utente), a gruppi di utenti o a tutti gli utenti.

a- Si modelli il problema descritto con un diagramma delle classi UML, evidenziando i metodi e gli attributi principali delle diverse classi.
b- In funzione delle classi identificate nel diagramma precedente, si scriva il corpo del metodo **stampaNomeCompleto** per scrivere a video il nome completo di un file partendo dalla radice del file system.
c- Si scriva l'invariante privato della classe **Directory** per dire che un suo oggetto non può mai contenere più di 200 file e deve esistere almeno un utente in grado di leggere il contenuto della directory.

NOTA: Se si ritiene che la descrizione informale contenga ambiguità, si descriva a parole come il diagramma UML proposto risolve tale ambiguità.

3.a



3.b

```
public String stampaNomeCompleto(){
    FSElement parent = getParent();
    if (parent != null){
        return parent.stampaNomeCompleto()+"/"+name;
    } else{
        return name;
    }
}
```

3.c

```
//@ private invariant
    \numof( int i; i > = 0 && i<= content.size();
          content.get(i) instanceof File) <= 200&&
    \exist(int j; j >= 0 && j< permissionOwners.size();
          permissionOwners.get(j) instanceof User &&
          permissionOwners.get(j).getPermissions().isRead());
```